Rotisserie auth story

Oct 10, 2026 · @Bradley Pettman

People sign in with email and password and belong to one household. Every request resolves to that household, and five separate checks keep it away from any other household's data. This is the design in PR #65, written up for review.

Decisions

Each choice favours what runs without an outside service, because the planned Lambda sits in a VPC with no outbound network.

Decision	Chose	Why	Passed on
Sign-in method	Email and password	No outside service; works in Expo Go today and on a Lambda with no internet access	Sign in with Apple and emailed one-time codes (both need outbound network), Cognito, Better Auth
Sessions	Random opaque token; only its SHA-256 is stored in Postgres	Revoked the moment a session row is deleted (sign-out) or a membership row is (removal); one lookup per request, and the request hits Postgres anyway	JWTs: valid until they expire, and need signing keys
Password hashing	scrypt from `node:crypto`	No native module to bundle for Lambda arm64; parameters on OWASP's list	Argon2 (native binary), bcrypt (ignores bytes past 72)
Households per person	One, as owner or member	The API derives the household from the session, so no URL carries a household id	Several, with a switcher in the app

Decision	Chose	Why	Passed on
Invites	Single-use 8-character codes, valid 7 days	Sent by message or read aloud; no email service needed	Email invitations, reusable links
Vocabulary	Tags per household; ingredients and units shared	Tags are personal; one dictionary keeps sharing recipes simple later, and listings hide other households' custom names	Everything shared (names leak), everything per household (seeds copied per household)
Enforcement	Composite foreign keys plus a household filter in every query	Postgres refuses a cross-household link even when a query forgets its filter	Row-level security: needs a per-request Postgres setting through the pool

Signing in and staying signed in

A session is a random token the phone keeps and the server only knows by its hash. It lasts 90 days from the last use.

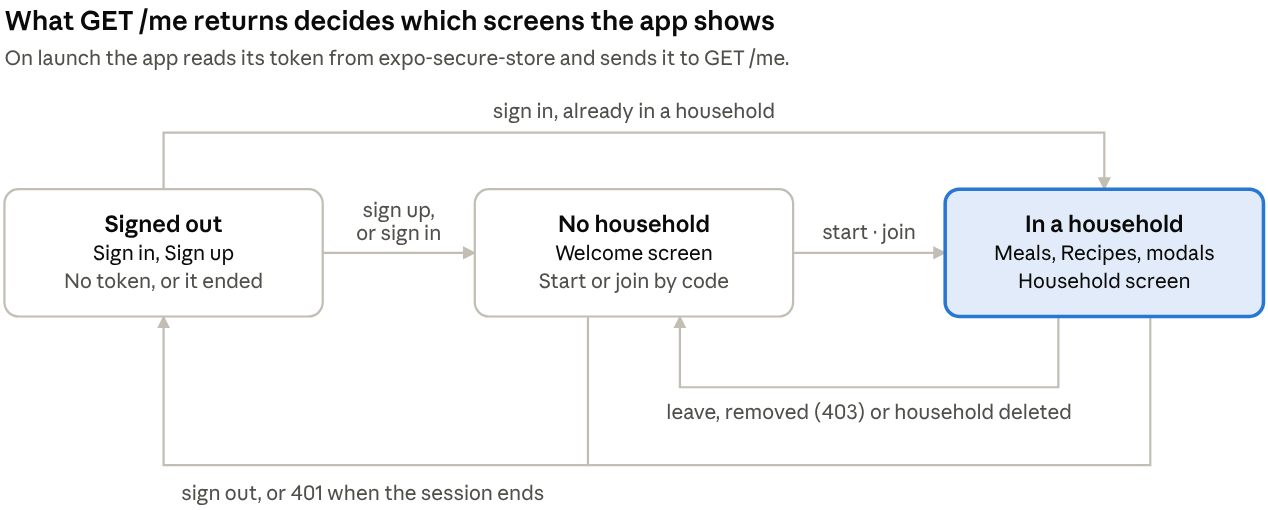
1. **Sign up or sign in.** `POST /auth/sign-up` stores an scrypt hash of the password. `POST /auth/sign-in` checks it, and gives a wrong password and an unknown email the same 401 and the same delay: when there is no account it still runs scrypt, on a throwaway hash.
2. **The server mints a session.** 32 random bytes, sent once as a 43-character token. Postgres keeps only its SHA-256 (`sessions.token_hash`), and a CHECK constraint rejects anything that isn't a hex hash, so the raw token can't be stored by mistake.
3. **The phone stores it in expo-secure-store.** That is Expo's wrapper over the iOS Keychain and Android's Keystore-backed storage: encrypted by the operating system, private to this app, and outside plain app storage such as AsyncStorage. The web build falls back to `localStorage`.
4. **Every request carries it** as `Authorization: Bearer <token>`. One query hashes the token, checks the session hasn't expired, and picks up the household and role.
5. **It slides forward.** When a session was last used more than a day ago, that request pushes its expiry to 90 days from now, so an app in use never signs out and a stale request doesn't cost a write.

6. **It ends three ways.** Sign-out deletes that session row and only that one. Expiry turns the next request into a 401. Being removed from a household leaves the session valid but turns data requests into 403 `no_household`.

Passwords need 8 to 256 characters. Emails are trimmed and lowercased before they're stored, and a CHECK constraint enforces it. The hash is a PHC string (`$scrypt$ln=15,r=8,p=3$salt$hash`: N = 32,768, about 32 MiB per check), so the parameters can be raised later without breaking stored hashes.

App states

The app shows one of three screen groups, picked by what `GET /me` says, and moves between them as the session and the membership change.



app states · 3 screen groups and the moves between them

Expo Router's `Stack.Protected` guards each group, so a screen outside the current state can't be opened. The splash screen stays up until the first answer, and settling past plans runs only for someone in a household. Signing out or switching accounts clears every cached query, so the next person never sees the last one's meals.

Households and roles

Everyone in a household shares one recipe book, one plan and one history. Only owners decide who is in it.

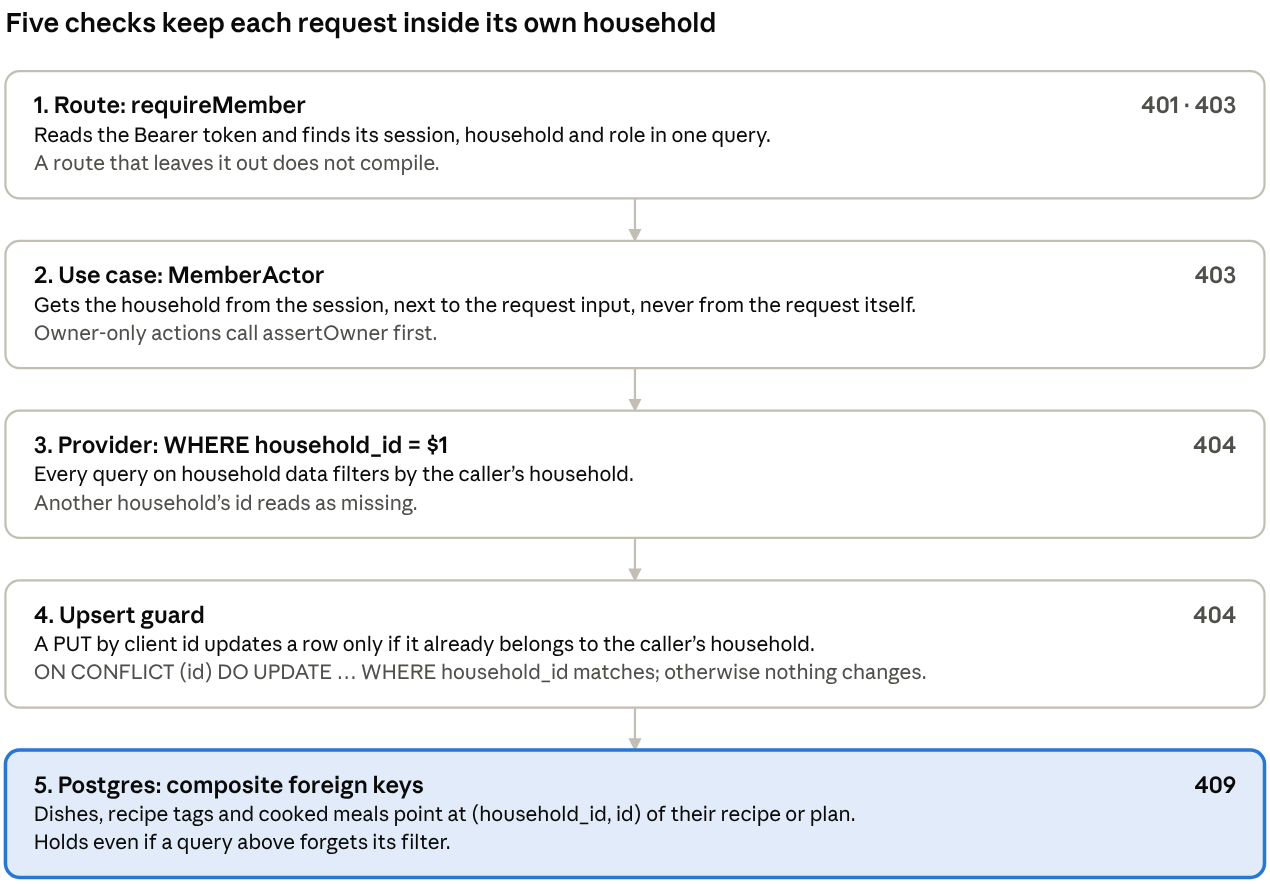
Action	Owner	Member
Read and edit recipes, plans and history	Yes	Yes
See the household and its members	Yes	Yes
Leave the household	Yes, unless they are the last owner	Yes
Rename the household	Yes	No (403)
Create, list and revoke invites	Yes	No (403)
Make someone an owner or a member	Yes	No (403)
Remove someone	Yes	No (403)
Delete the household and all its data	Yes	No (403)

The rules that keep a household usable:

- A household always has an owner. Demoting or removing the last owner is refused with a 409. When the last owner is also the only member, the app offers Delete household instead.
- A person belongs to at most one household (a unique `user_id` on `household_members`). Joining another means leaving first.
- Invite codes are 8 characters from a 30-character alphabet without look-alikes (no 0/O, 1/I/L or U), about 39 bits. Each works once, lasts 7 days, and is shown as `K7QM-2XPA`. Codes are stored as plain text so owners can share them again from the Household screen.
- The first person to join a household with no members becomes its owner. This is how the household the migration creates for existing data gets its owner.

How a request stays inside its household

Five checks stand between a request and another household's rows. The last one is in Postgres, so it holds even if the code above it has a bug.



request checks · 5 layers, top to bottom

The numbers on the right are what each check returns. `isolation.test.ts` signs in as a second household and tries to read, overwrite, delete and link to the first household's data. Removing check 3 from settle, or check 4 from the recipe upsert, makes those tests fail.

Data model

Three migrations add five tables and a `household_id` on seven existing ones. Existing rows move into a household called "Home".

Table	Holds	Rules that matter
`users`	Email, display name, scrypt hash	Email unique, lowercase and trimmed (CHECK)

Table	Holds	Rules that matter
`sessions`	One row per signed-in device	`token_hash` unique and hex-only (CHECK); deleted with its user
`households`	Name	Deleting one deletes all its recipes, tags, plans and history
`household_members`	Who is in which household, as `owner` or `member`	`user_id` unique: one household per person
`household_invites`	Code, expiry, who created it	Code unique, 8 characters from the invite alphabet (CHECK); deleted when used
`recipes`, `tags`, `planned_meals`, `cooked_meals`	Gain `household_id`	`ON DELETE CASCADE`; tag names unique per household; one meal per slot per household
`recipe_tags`, `planned_meal_dishes`, `cooked_meal_dishes`	Gain `household_id`	Both foreign keys include it, so each row's parents share its household
`ingredients`, `units`	Gain `is_standard`	Stay shared; the seeds mark their rows standard

The history rule from before still holds: deleting a recipe deletes its planned dishes but only clears `recipe_id` on cooked dishes, now written `ON DELETE SET NULL (recipe_id)` so the household column stays.

API and errors

Every existing route now requires a household member; `/health` stays public. These routes are new:

Route	Who can call it	Does
`POST /auth/sign-up`	Anyone	Creates an account and a session (201)
`POST /auth/sign-in`	Anyone	Creates a session
`POST /auth/sign-out`	Signed in	Ends this session (204)
`GET /me`	Signed in	The user and their household, or `household: null`
`POST /household`	Signed in, no household	Starts one, with the caller as owner (201)
`POST /household/join`	Signed in, no household	Joins with an invite code
`GET /household`	Member	Name and members
`PATCH /household`	Owner	Renames it
`DELETE /household`	Owner	Deletes it and everything in it (204)
`GET` · `POST /household/invites`	Owner	Lists or creates invites
`DELETE /household/invites/:id`	Owner	Revokes an invite (204)
`PATCH /household/members/:userId`	Owner	Makes someone an owner or a member
`DELETE /household/members/:userId`	Owner, or the member themselves	Removes them, or lets them leave (204)

The errors use the existing envelope, `{ error: { code, message, fields? } }`:

Status	Code	Means	The app
401	`unauthenticated`	No token, an unknown one, or an expired session; also a wrong password	Clears the token and caches, shows Sign in; on the sign-in screen itself, shows the message

Status	Code	Means	The app
403	`no_household`	Signed in but not in a household, for example just removed	Refetches `/me` and shows Welcome
403	`forbidden`	A member tried an owner action	Shows the message
404	`not_found`	Missing, or belongs to another household; also a wrong or expired invite code	Shows the message
409	`conflict`	Email taken, already in a household, last owner, or a link to another household's row	Shows the message

Known gaps

None of these block using the app at home; the first two should land before it goes on the internet.

Gap	What could go wrong	When it matters
No rate limit on sign-in or joining	Unlimited password guesses against one account. Guessing an invite code is impractical at 39 bits	Before deploy; API Gateway route throttling is enough to start
No password change or reset	A forgotten password locks that person out. Reset needs email, and the VPC Lambda has no outbound network	Before family members rely on it
No account deletion	Apple requires apps that let people create accounts to let them delete them in the app	Before App Store review

Gap	What could go wrong	When it matters
No email verification	Someone can register an address that isn't theirs, which keeps its owner from signing up	Once reset by email exists
Sign-up says when an email is already registered	Reveals who has an account (409 instead of a generic answer)	Low for a family app
No list of active sessions or "sign out everywhere"	A lost phone stays signed in: nobody can end its session from another device, and each use pushes the expiry out again	Later; the sessions table already supports it
Development runs over plain HTTP on the LAN	The token can be read on the home network	Only in development; the deployed API is HTTPS through API Gateway
Web build keeps the token in `localStorage`, and the API sends no CORS headers	Script injected into the page could read the token; the web build can't reach the API yet	Only if a web client becomes real

Questions for you

Each has a default already built; tick the ones you're happy with and comment on the rest.

- [x] ~~**Only owners invite.** Should members be able to create invite codes too?~~
- [x] ~~**Sessions last 90 days from last use.** Shorter, longer, or no expiry for a family app?~~
- [ ] **Codes are stored as plain text** so owners can share them again. Hash them instead and show each code only once?
- [x] ~~**Delete household appears only when you're the last member**, though the API lets any owner delete it. Offer it to owners of a shared household too?~~
- [ ] **The invite script** (`npm run invite`) exists only to claim "Home" after migrating. Delete it once your database is migrated?

- [ ] **Rate limiting waits for deploy** (API Gateway throttling). Add a simple per-email limit in the API now instead?